

StarForth VM and Physics-Driven Adaptive Runtime

Volume I of the StarshipOS Technical Reference

Robert A. James

Contents

1	Introduction	5
1.1	What StarForth Is	5
1.2	Technology Stack	5
1.3	Thermodynamic Modeling Language	5
1.4	Document Scope and Organization	5
2	FORTH-79 Interpreter	7
2.1	Memory Model	7
2.2	Dictionary Structure	7
2.3	FORTH-79 Word Categories	7
2.4	Initialization and Boot	7
2.5	Q48.16 Fixed-Point Arithmetic	7
3	Physics-Driven Adaptive Runtime	9
3.1	Overview: Seven Feedback Loops	9
3.2	Loop #1 — Execution Heat	9
3.3	Loop #2 — Rolling Window of Truth	9
3.4	Loop #3 — Linear Decay	9
3.5	Loop #4 — Word Transition Prediction (Pipelining)	9
3.6	Loop #5 — Window Width Inference	9
3.7	Loop #6 — Decay Slope Inference	9
3.8	Loop #7 — Adaptive Heartrate	9
3.9	L8 Jacquard Mode Selector	9
3.10	Determinism and Formal Properties	9
4	Formal Verification	11
4.1	Proof Infrastructure	11
4.2	Base Definitions	11
4.3	Physics Loop Proofs	11
4.3.1	Loop #1: Execution Heat	11
4.3.2	Loop #2: Rolling Window Bounded Growth	11
4.3.3	Loop #3: Decay Convergence	11
4.3.4	Loop #4: Pipelining Metrics Safety	11
4.3.5	Loop #5: Window Width Inference Termination	11
4.3.6	Loop #6: Decay Slope Inference Soundness	11
4.3.7	Loop #7: Heartrate Adaptive Stability	11
4.4	Word-Category Proofs	11
4.5	Concurrency and Mutex	11
4.6	Overall Correctness	11
4.7	ACL System Proofs	11

5	Experimental Results	13
5.1	Experimental Design	13
5.2	Primary Result: Steady-State Convergence	13
5.3	Claim Table	13
5.4	Negative Results and Limitations	13
5.5	Sensitivity Analysis	13
5.6	Reproducing the Results	13
6	Build and Deployment	15
6.1	Prerequisites	15
6.2	Build Targets	15
6.3	Build Flags	15
6.4	Running	15
6.5	Test Suite	15
7	Word-Level ACL System	17
7.1	Design Summary	17
7.2	Phase Status	17
7.3	ACL Formal Proofs	17

Chapter 1

Introduction

1.1 What StarForth Is

1.2 Technology Stack

Table 1.1: StarshipOS technology stack

Layer	Component
Future OS	StarshipOS (self-hosting)
Kernel	LithosAnanke v1.0.8 / StarKernel (bare-metal UEFI)
VM	StarForth v3.0.3 (FORTH-79, physics-driven adaptive RT)
Host platform	Linux, L4Re/Fiasco.OC, bare metal (amd64, aarch64, riscv64)

1.3 Thermodynamic Modeling Language

1.4 Document Scope and Organization

Chapter 2

FORTH-79 Interpreter

2.1 Memory Model

2.2 Dictionary Structure

2.3 FORTH-79 Word Categories

Table 2.1: StarForth word implementation files

File	Scope
arithmetic_words.c	+ - * / MOD ABS MIN MAX
stack_words.c	DUP DROP SWAP ROT OVER NIP TUCK
control_words.c	IF ELSE THEN DO LOOP BEGIN UNTIL WHILE
defining_words.c	: ; CREATE DOES> VARIABLE CONSTANT
memory_words.c	@ ! C@ C! MOVE FILL
return_stack_words.c	>R R> R@ RDROP 2>R 2R@ 2R>
double_words.c	2DUP 2DROP 2SWAP 2@ 2! D+ D-
logical_words.c	AND OR XOR NOT INVERT LSHIFT RSHIFT
io_words.c	EMIT KEY TYPE CR TAB SPACE ACCEPT
string_words.c	S" SLITERAL and string operations
block_words.c	BLOCK BUFFER LOAD THRU FLUSH
format_words.c	.(.R .S HEX DECIMAL BASE
system_words.c	BYE ABORT INCLUDE STATE
dictionary_words.c	FIND SEARCH-WORDLIST WORDS
vocabulary_words.c	VOCABULARY DEFINITIONS FORTH-WORDLIST
q48_16_words.c	Q _{48.16} fixed-point word definitions
starforth_words.c	StarForth-specific extensions

2.4 Initialization and Boot

2.5 Q48.16 Fixed-Point Arithmetic

Chapter 3

Physics-Driven Adaptive Runtime

- 3.1 Overview: Seven Feedback Loops
- 3.2 Loop #1 — Execution Heat
- 3.3 Loop #2 — Rolling Window of Truth
- 3.4 Loop #3 — Linear Decay
- 3.5 Loop #4 — Word Transition Prediction (Pipelining)
- 3.6 Loop #5 — Window Width Inference
- 3.7 Loop #6 — Decay Slope Inference
- 3.8 Loop #7 — Adaptive Heartrate
- 3.9 L8 Jacquard Mode Selector
- 3.10 Determinism and Formal Properties

Chapter 4

Formal Verification

4.1 Proof Infrastructure

4.2 Base Definitions

4.3 Physics Loop Proofs

4.3.1 Loop #1: Execution Heat

4.3.2 Loop #2: Rolling Window Bounded Growth

4.3.3 Loop #3: Decay Convergence

4.3.4 Loop #4: Pipelining Metrics Safety

4.3.5 Loop #5: Window Width Inference Termination

4.3.6 Loop #6: Decay Slope Inference Soundness

4.3.7 Loop #7: Heartrate Adaptive Stability

4.4 Word-Category Proofs

4.5 Concurrency and Mutex

4.6 Overall Correctness

4.7 ACL System Proofs

Chapter 5

Experimental Results

5.1 Experimental Design

5.2 Primary Result: Steady-State Convergence

5.3 Claim Table

5.4 Negative Results and Limitations

5.5 Sensitivity Analysis

5.6 Reproducing the Results

Chapter 6

Build and Deployment

6.1 Prerequisites

6.2 Build Targets

Listing 6.1: StarForth build targets

```
1 make          # Standard optimized build (auto-detects
   architecture)
2 make fastest  # Maximum performance (ASM + LTO + direct
   threading)
3 make pgo      # Profile-guided optimization
4 make debug    # Debug build with symbols
5 make test     # Full test suite (936+ tests)
6 make bench    # Quick benchmark
7 make clean    # Remove build artifacts
```

6.3 Build Flags

6.4 Running

Listing 6.2: Running StarForth

```
1 ./build/amd64/standard/starforth      # Interactive REPL
2 ./build/amd64/standard/starforth --run-tests # Tests then REPL
3 ./build/amd64/standard/starforth -c "1_2_+_.BYE" # Inline code
4 ./build/amd64/fastest/starforth --doe    # DoE mode
```

6.5 Test Suite

Chapter 7

Word-Level ACL System

7.1 Design Summary

The word-level ACL system is implemented through Phase 6. Key constraints (cite the design doc for full details):

- All policy logic in `ACL.4th` — no new C primitives for policy.
- Four DictEntry C fields: `acl_ttl`, `acl_allow`, `acl_mode`, `acl_pinned`.
- Two VM flags: `emergency_console` (fault handler active) and `zuse_session` (superuser authenticated).
- `ACL.4th` is self-activating; `init.4th` only needs `S" ACL.4th" EXEC`.
- ACL-PIN is one-way; inheritance clears pin, copies mode.

7.2 Phase Status

Table 7.1: ACL implementation phase status

Phase	Description	Status
1	C Infrastructure (DictEntry fields, interpreter hook, <code>acl_recheck()</code>)	Done
2	<code>ACL.4th</code> FORTH policy words + <code>ACL-INIT-PRIMITIVES</code> + self-activation	Done
3	<code>capsules/zuse.4th</code> bootstrap superuser skeleton; CA root placeholder	Done
4	<code>init.4th</code> opt-in toggle	Done
5	POST tests (800/800) + Isabelle/HOL proofs (5 theory files)	Done
6	<code>EMERGENCY_CONSOLE_ENABLED</code> build flag + <code>vm_fault_handler</code>	Done
7	LithosAnanke parity — port to kernel context, three-arch acceptance	Pending
8	PKI / thumbdrive — Ed25519 challenge-response; user minting by zuse	Future

7.3 ACL Formal Proofs

Five Isabelle/HOL theorems cover the ACL system’s core safety properties:

1. **Pin Monotonicity** (`ACL_Pin_Monotone.thy`) — once pinned, a word remains pinned through the system’s lifetime.

2. **Inheritance Clears Pin** (`ACL_Inherit_Clears_Pin.thy`) — inherited words carry mode but never carry pin status.
3. **TTL Bounded** (`ACL_TTL_Bounded.thy`) — all TTL values remain within defined bounds; no overflow.
4. **Emergency Bypass Correct** (`ACL_Emergency_Bypass.thy`) — the emergency console bypass does not escalate privileges.
5. **No Escalation** (`ACL_No_Escalation.thy`) — no sequence of ACL operations allows a word to acquire permissions it was not granted.